A PROPOSAL FOR THE LEADERSHIP TEAM

Six Capabilities Our Current Architecture Cannot Deliver

An honest case for moving to event-sourced architecture, grounded in concrete business outcomes.

THE PROBLEM

Questions Our Systems Cannot Answer Today

Each of these is a real request we have received from the business in the last year.

FROM COMPLIANCE

"Show me everything that happened to this customer's account on March 14, in chronological order."

Today's answer: a multi-day investigation across logs and DBs.

FROM PRODUCT

"What would our churn rate have looked like last quarter under the new definition?"

Today's answer: not possible. We don't store the inputs.

FROM DATA SCIENCE

"Can we train a model on three years of customer behavior, not just the current snapshot?"

Today's answer: we have snapshots, not behavior history.

FROM CUSTOMER SUCCESS

"Why did this customer's plan change three times last week? What did they do?"

Today's answer: we see the current state, not the journey.

The pattern: our systems record current state.
The business increasingly needs *history.*

CAPABILITY 1 OF 6

100% Audit Log

Every change traceable to who did it, when, and why. Without effort.

WHAT THIS MEANS

Instead of recording the current state of accounts, orders, and customers, the system records every change as it happens. Nothing is overwritten. The history is the data, not a side effect of it.

WHAT IT ENABLES

Regulatory response

Auditor requests answered in minutes, not days. The audit log is the system itself.

Customer support

"Why did my account get charged twice?" answered with the actual sequence of events.

Bug investigation

"What was the system seeing when it made that decision?" becomes a query, not a guess.

CONCRETE EXAMPLE

"When the auditor asked why account X was charged twice on March 14, we answered in 5 minutes with the full sequence of events. Not 3 days of log spelunking."

CAPABILITY 2 OF 6

Time-Travel

Reconstruct the exact state of any account, order, or system at any past moment.

WHAT THIS MEANS

Because the system stores events rather than overwriting state, we can replay events up to any point in time and see exactly what the system looked like then. The past is never lost.

WHAT IT ENABLES

Historical reporting

"What would last quarter look like under our new churn definition?" Answerable.

Post-incident reconstruction

Replay events to the moment of the incident; see exactly what the system was doing.

New metrics, old data

Define a metric today, compute it back across years of history. No "we don't have that data."

CONCRETE EXAMPLE

"Marketing wanted to know how the new pricing tier would have performed if launched a year earlier. We replayed the events with the new logic. Had the answer the next day."

CAPABILITY 3 OF 6

AI-Ready Data

Every system becomes its own training dataset, growing more valuable each year.

WHAT THIS MEANS

Machine learning models need behavior over time, not snapshots. An event-sourced system records every customer interaction in sequence. Exactly what ML models need to predict the next one.

WHAT IT ENABLES

Train on real behavior

Three years of customer events, not three years of nightly state snapshots.

Retrain anytime

New model architecture? Replay the event log to retrain. The data is always available.

Deterministic backtesting

Test new models against the exact historical data. No drift. between training and reality.

CONCRETE EXAMPLE

"The data science team built a churn-prediction model using two years of event history we already had. With our previous architecture, they would have started from scratch. Collecting data first."

CAPABILITY 4 OF 6

Many Views, One Truth

The same source data feeds many products, each optimized for its own purpose.

WHAT THIS MEANS

One event log feeds many purpose-built views. Marketing gets analytical dashboards, customer success gets operational tooling, data science gets training data. All from the same source of truth.

WHAT IT ENABLES

No data silos

Marketing's numbers and customer success's numbers come from the same source.

New views in days

A team needs a new dashboard? Build a projection. No database migrations. No coordination.

Compounding leverage

Each new use of the data adds value without changing the source. The asset compounds.

CONCRETE EXAMPLE

"Finance and product had different revenue numbers for two years. With one event log feeding both teams' dashboards, the discrepancy disappeared. There is only one number now, by construction."

CAPABILITY 5 OF 6

Rewrite Avoidance

Continuous evolution, not periodic 18-month "version 2.0" rewrites.

WHAT THIS MEANS

New requirements typically force schema migrations, data backfills, or full rewrites. Event-sourced systems handle change by adding new event types or new projections, alongside what already runs.

WHAT IT ENABLES

New fields, no migration

Add a new event field. The old events keep working. No downtime. No data backfill.

New integrations, no risk

A new partner needs our data? They subscribe to events. The core system doesn't change.

Avoid the big-bang project

No more 18-month "version 2" rewrites. Evolution happens in weekly, not annual, increments.

CONCRETE EXAMPLE

"We added a new revenue category mid-year. In the old system, that would have been a database migration affecting 12 reports. With events, it was a new projection. Done in a sprint, no downtime."

CAPABILITY 6 OF 6

Compliance Hedge

Survives regulatory questions you cannot anticipate today.

WHAT THIS MEANS

Future regulations will ask for data we did not plan to keep. Future audits will ask questions we did not anticipate. Event sourcing is insurance: the data is already captured, even if we did not know why.

WHAT IT ENABLES

Future audits answerable

When the regulator asks for data from three years ago, the answer is "yes." Without scrambling.

New regulations, no panic

A new privacy or financial rule requires reporting on past data? Build the report. Data is there.

Stronger negotiation

In disputes with customers, vendors, or regulators, the full history is on our side.

CONCRETE EXAMPLE

"A new compliance requirement asked for retroactive reporting on customer consent changes. Three years of consent events were already in the log. Competitors spent months retrofitting; we shipped."

CREDIBILITY CHECK

What This Is Not

An honest list of what we are not promising. Everything else loses credibility.

Not a rewrite.

We do not propose throwing away the existing systems. Event sourcing is added one bounded context at a time, alongside what already runs.

The big-bang rewrite is specifically what this avoids.

Not a 2-year project.

First production deployment in 6 months. Broader adoption over 12-18 months. Each step delivers value; nothing waits.

No "trust us, the value arrives at the end."

Not for every system.

CRUD-by-nature systems should stay CRUD. Prototypes should stay simple. Not every problem benefits from this.

We will name specifically which systems should adopt.

SYSTEMS WHERE WE WILL NOT USE EVENT SOURCING

- Internal tools where the history is uninteresting (brochure managers, simple admin panels)
- Throwaway prototypes. Rapid CRUD iteration is faster for unknown requirements
- Pure data pipelines (ETL, stream processing). Purpose-built tools serve these better
- Systems with a single dominant query (search indexes, recommendation engines)
- Compliance environments that require provable, auditable data *deletion* (some privacy regimes)

THE BUDGETING CONVERSATION

What It Costs

Real numbers, not vague reassurances. We have done the arithmetic.

COST CATEGORY	ESTIMATE	WHEN
Team ramp-up time Two to six weeks to productivity per engineer	25-30 engineer-weeks	Q1 only
Contractor / specialist External event-sourcing expertise for the first quarter	~$60-80K	Q1-Q2
Storage Event log + projections at our event volume	~$300-500/year	Ongoing
On-call expansion Additional infrastructure to monitor (event store, projections)	Modest	Ongoing

Total first-year cost: well under one engineer-year of fully-loaded cost.
By comparison: the next "version 2.0" rewrite would cost 5-10× this.

THE PLAN

How We Would Do It

One bounded context first. Strangler pattern. Each step delivers value on its own.

MONTH 1-3

Discovery

· Event-storm one bounded context (suggest: billing or audit)
· Engineer training

MONTH 4-6

First Production

· Build the chosen bounded context
· Ship to production
· Run alongside existing systems

MONTH 7-12

Evaluate & Expand

· Measure value vs cost honestly
· Decide: expand or stop?
· Add second context

MONTH 13+

Steady Adoption

· One context per quarter
· Old systems shrink naturally
· No "switchover day"

EXIT RAMPS BUILT IN

After Month 6, we evaluate: did the first bounded context deliver the promised value? If yes, expand.
If no, we stop with one new system and the lessons learned. *No commitment to the full migration up front.*

THE ASK

The Decision

Three approvals would unblock the discovery quarter. We need a decision by [date].

- [ ] **Approve the discovery quarter**
  Q1 spend: ~25-30 engineer-weeks of focused time on event-storming and design.

- [ ] **Approve contractor budget**
  ~$60-80K for an external event-sourcing specialist embedded in the team for Q1-Q2.

- [ ] **Approve the first bounded context**
  Recommend: billing or audit. Both deliver compliance value in Month 6 even if we stop there.

Decision needed by [date]

Next step: 30-minute follow-up to assign the discovery team.